

2.2. PROGRAMACIÓN MODULAR

E.S

La *programación modular* es uno de los métodos de diseño más flexible y potentes para mejorar la productividad de un programa. En programación modular el programa se divide en *módulos* (partes independientes), cada una de las cuales ejecuta una única actividad o tarea y se codifican independientemente de otros módulos. Cada uno de estos módulos se analizan, codifican y ponen a punto por separado.

Cada programa contiene un módulo denominado *programa principal*, que controla todo lo que sucede; se transfiere el control a *submódulos* (posteriormente se denominarán *subprogramas*), de modo que ellos puedan ejecutar sus funciones; sin embargo, cada submódulo devuelve el control al módulo principal cuando se haya completado su tarea. Si la tarea asignada a cada submódulo es demasiado compleja, éste deberá romperse en otros módulos más pequeños. El proceso sucesivo de subdivisión de módulos continúa hasta que cada módulo tenga solamente una tarea específica que ejecutar. Esta tarea puede ser *entrada, salida, manipulación de datos, control de otros módulos* o alguna *combinación de éstos*. Un módulo puede transferir temporalmente (*bifurcar*) el control a otro módulo; sin embargo, cada módulo debe eventualmente devolver el control al módulo del cual se recibe originalmente el control.

Los módulos son independientes en el sentido en que ningún módulo puede tener acceso directo a cualquier otro módulo excepto el módulo al que llama y sus propios submódulos. Sin embargo, los resultados producidos por un módulo pueden ser utilizados por cualquier otro módulo cuando se transfiera a ellos el control.

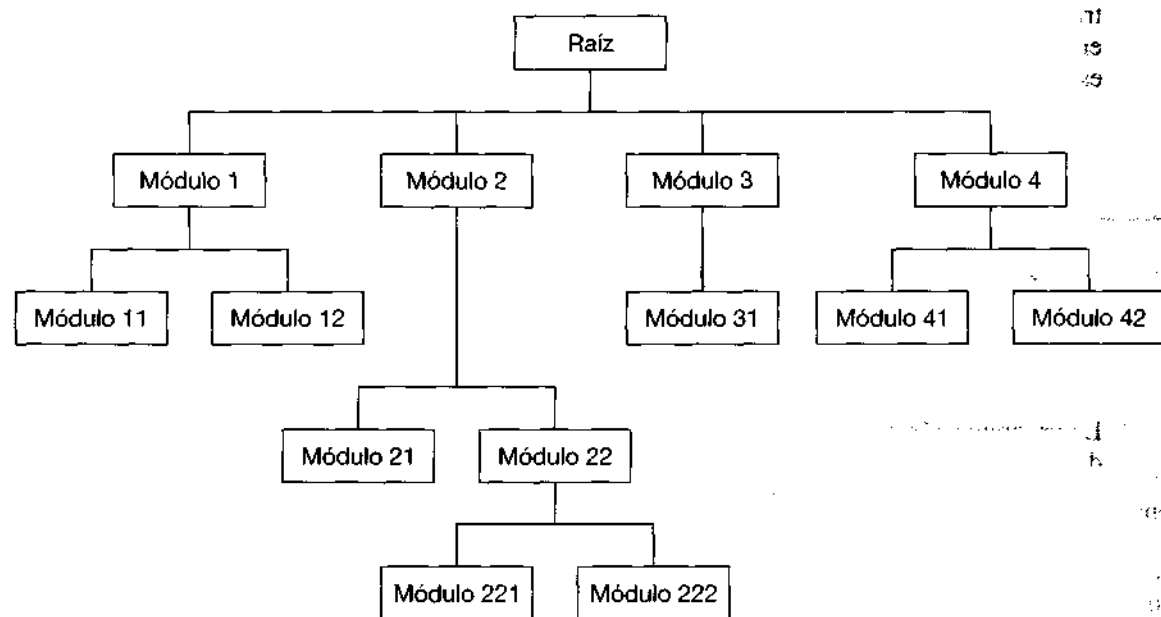


Figura 2.6. Programación modular.

b oñeziD .S.S.S

Dado que los módulos son independientes, diferentes programadores pueden trabajar simultáneamente en diferentes partes del mismo programa. Esto reducirá el tiempo del diseño del algoritmo y posterior codificación del programa. Además, un módulo se puede modificar radicalmente sin afectar a otros módulos, incluso sin alterar su función principal.

La descomposición de un programa en módulos independientes más simples se conoce también como el método de «*divide y vencerás*» (*divide and conquer*). Se diseña cada módulo con independencia de los demás, y siguiendo un método ascendente o descendente se llegará hasta la descomposición final del problema en módulos en forma jerárquica.

SUBPROGRAMAS (SUBALGORITMOS): Procedimientos y funciones

CONTENIDO

- | | |
|---|--|
| 6.1. Introducción a los subalgoritmos o subprogramas. | 6.7. Los efectos laterales. |
| 6.2. Funciones. | 6.8. Recursión (recursividad). |
| 6.3. Procedimientos (subrutinas). | ACTIVIDADES DE PROGRAMACION RESUELTAS. |
| 6.4. Ámbito: variables locales y globales. | REVISIÓN DEL CAPÍTULO. |
| 6.5. Comunicación con subprogramas: paso de parámetros. | Conceptos clave. |
| 6.6. Funciones y procedimientos como parámetros. | Resumen. |
| | EJERCICIOS. |

La resolución de problemas complejos se facilita considerablemente si se dividen en problemas más pequeños (subproblemas). *La solución de estos subproblemas se realiza con subalgoritmos.* El uso de subalgoritmos permite al programador desarrollar programas de problemas complejos utilizando el método descendente introducido en los capítulos anteriores. *Los subalgoritmos (subprogramas) pueden ser de dos tipos: funciones y procedimientos o subrutinas.* Los subalgoritmos son unidades de programa o módulos que están diseñados para ejecutar alguna tarea específica. Estas funciones y procedimientos se escriben solamente una vez, pero pueden ser referenciados en diferentes puntos de un programa, de modo que se puede evitar la duplicación innecesaria del código.

Las unidades de programas en el estilo de programación modular son independientes; el programador puede escribir cada módulo y verificarlo sin preocuparse de los detalles de otros módulos. Esto facilita considerablemente la localización de un error cuando se produce. Los programas desarrollados de este modo son normalmente también más fáciles de comprender, ya que la estructura de cada unidad de programa puede ser estudiada independientemente de las otras unidades de programa. En este capítulo se describen las *funciones y procedimientos*, junto con los conceptos de *variables locales y globales*, así como *parámetros*. Se introduce también el concepto de *recursividad* como una nueva herramienta de resolución de problemas.

6.1. INTRODUCCIÓN A LOS SUBALGORITMOS O SUBPROGRAMAS

Un método ya citado para solucionar un problema complejo es dividirlo en subproblemas —problemas más sencillos— y a continuación dividir estos subproblemas en otros más simples, hasta que los problemas más pequeños sean fáciles de resolver. Esta técnica de dividir el problema principal en subproblemas se suele denominar «*divide y vencerás*» (*divide and conquer*). Este método de diseñar la solución de un problema principal obteniendo las soluciones de sus subproblemas se conoce como *diseño descendente* (*top-down design*). Se denomina descendente, ya que se inicia en la parte superior con un problema general y el diseño específico de las soluciones de los subproblemas. Normalmente las partes en que se divide un programa deben poder desarrollarse independientemente entre sí.

Las soluciones de un diseño descendente pueden implementarse fácilmente en lenguajes de programas de alto nivel, como C/C++, Pascal o FORTRAN. Estas partes independientes se denominan *subprogramas* o *subalgoritmos* si se emplean desde el concepto algorítmico.

La correspondencia entre el diseño descendente y la solución por computadora en términos de programa principal y sus subprogramas se analizará a lo largo de este capítulo.

Consideremos el problema del cálculo de la superficie (área) de un rectángulo. Este problema se puede dividir en tres subproblemas:

```
subproblema 1: entrada de datos de altura y base.
subproblema 2: cálculo de la superficie.
subproblema 3: salida de resultados.
```

El algoritmo correspondiente que resuelve los tres *subproblemas* es:

```
leer (altura, base)           //entrada de datos
area ← base * altura         //cálculo de la superficie
escribir(base, altura, area) //salida de resultados
```

El método descendente se muestra en la Figura 6.1.

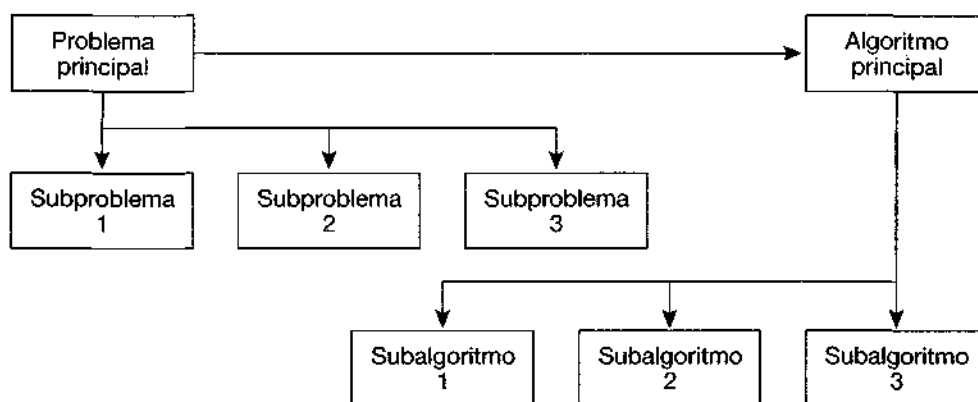


Figura 6.1. Diseño descendente.

El problema principal se soluciona por el correspondiente **programa** o **algoritmo principal** —también denominado *controlador* o *conductor* (*driver*)— y la solución de los subproblemas mediante **subprogramas**, conocidos como **procedimientos** (*subrutinas*) o **funciones**. Los subprogramas, cuando se tratan en lenguaje algorítmico, se denominan también *subalgoritmos*.

Un subprograma puede realizar las mismas acciones que un programa: 1) aceptar datos, 2) realizar algunos cálculos y 3) devolver resultados. Un subprograma, sin embargo, se utiliza por el progra-

ma para un propósito específico. El subprograma recibe datos desde el programa y le devuelve resultados. Haciendo un símil con una oficina, el problema es como el jefe que da instrucciones a sus subordinados —subprogramas—; cuando la tarea se termina, el subordinado devuelve sus resultados al jefe. Se dice que el programa principal *llama* o *invoca* al subprograma. El subprograma ejecuta una tarea, a continuación *devuelve* el control al programa. Esto puede suceder en diferentes lugares del programa. Cada vez que el subprograma es llamado, el control retorna al lugar desde donde fue hecha la llamada (Figura 6.2). Un subprograma puede llamar a su vez a sus propios subprogramas (Figura 6.3). Existen —como ya se ha comentado— dos tipos importantes de subprogramas: *funciones* y *procedimientos* o *subrutinas*.

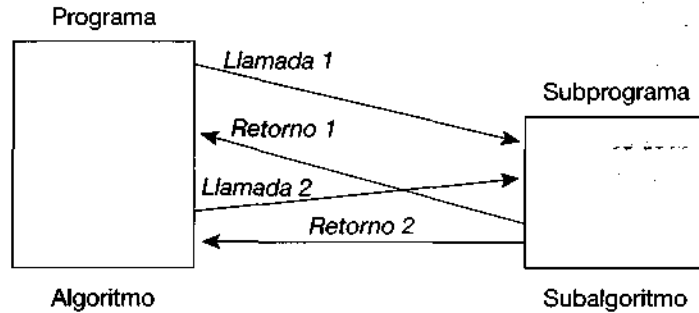


Figura 6.2. Un programa con un subprograma: función y procedimiento o subrutina, según la terminología específica del lenguaje: subrutina en BASIC y FORTRAN, *función* en C, C++, método en Java o C#, *procedimiento* o *función* en Pascal.

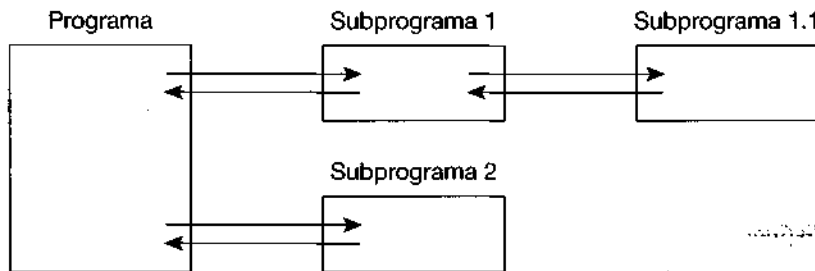


Figura 6.3. Un programa con diferentes niveles de subprogramas.

6.2. FUNCIONES

Matemáticamente una función es una operación que toma uno o más valores llamados *argumentos* y produce un valor denominado *resultado* —valor de la función para los argumentos dados—. Todos los lenguajes de programación tienen funciones incorporadas o intrínsecas —en el Capítulo 2 se vieron algunos ejemplos— y funciones definidas por el usuario. Así, por ejemplo

$$f(x) = \frac{x}{1+x^2}$$

donde f es el nombre de la función y x es el argumento. Obsérvese que ningún valor específico se asocia con x ; es un *parámetro formal* utilizado en la definición de la función. Para evaluar f debemos dar-

le un valor real o actual a x ; con este valor se puede calcular el resultado. Con $x = 3$ se obtiene el valor 0.3 que se expresa escribiendo

$$f(3) = 0.3$$

$$f(3) = \frac{3}{1+9} = \frac{3}{10} = 0.3$$

Una función puede tener varios argumentos. Por consiguiente,

$$f(x, y) = \frac{x-y}{\sqrt{x}-\sqrt{y}}$$

es una función con dos argumentos. Sin embargo, solamente un único valor se asocia con la función para cualquier par de valores dados a los argumentos.

Cada lenguaje de programación tiene sus propias funciones incorporadas, que se utilizan escribiendo sus nombres con los argumentos adecuados en expresiones tales como

```
raiz2(A+cos(x))
```

Cuando la expresión se evalúa, el valor de x se da primero al subprograma (función) coseno y se calcula $\cos(x)$. El valor de $A+\cos(x)$ se utiliza entonces como argumento de la función *raiz2* (raíz cuadrada), que evalúa el resultado final.

Cada función se evoca utilizando su nombre en una expresión con los argumentos actuales o reales encerrados entre paréntesis.

Las funciones incorporadas al sistema se denominan *funciones internas o intrínsecas* y las funciones definidas por el usuario, *funciones externas*. Cuando las funciones estándares o internas no permiten realizar el tipo de cálculo deseado es necesario recurrir a las funciones externas que pueden ser definidas por el usuario mediante una *declaración de función*.

A una función no se le llama explícitamente, sino que se le invoca o referencia mediante un nombre y una lista de parámetros actuales. El algoritmo o programa llama o invoca a la función con el nombre de esta última en una expresión seguida de una lista de argumentos que deben coincidir en cantidad, tipo y orden con los de la función que fue definida. La función devuelve un único valor.

Las funciones son diseñadas para realizar tareas específicas: tomar una lista de valores —llamados *argumentos*— y devolver un único valor.

6.2.1. Declaración de funciones

La declaración de una función requiere una serie de pasos que la definen. Una función como tal subalgoritmo o subprograma tiene una constitución similar a los algoritmos, por consiguiente, constará de una cabecera que comenzará con el tipo del valor devuelto por la función, seguido de la palabra **función** y del nombre y argumentos de dicha función. A continuación irá el *cuerpo* de la función, que será una serie de acciones o instrucciones cuya ejecución hará que se asigne un valor al nombre de la función. Esto determina el valor particular del resultado que ha de devolverse al programa llamador.

La declaración de la función será;

```
<tipo_de_resultado> funcion <nombre_fun> (lista de parametros)
[declaraciones locales]
inicio
    <acciones>           //cuerpo de la funcion
    devolver (<expresion>)
fin_función
```

lista de parametros lista de *parámetros formales* o *argumentos*, con uno o más argumentos de la siguiente forma:
 ({E|S|E/S} tipo_de_datoA: parámetro 1[, parámetro 2]...; ;
 {E|S|E/S} tipo_de_datoB: parámetro x[, parámetro y]...)

nombre_func nombre asociado con la función, que será un nombre de identificador válido

<acciones> instrucciones que constituyen la definición de la función y que debe contener una única instrucción: **devolver** (<expresion>); expresión sólo existe si la función se ha declarado con valor de retorno y *expresión* en el valor devuelto por la función.

tipo_de_resultado tipo del resultado que devuelve la función

Invocación .S.S.8

Sentencia **devolver** (return)

La sentencia **devolver** (**return**, volver) se utiliza para regresar de una función (un *método* en programación orientada a objetos); **devolver** hace que el control del programa se transfiera al llamador de la función (método). Esta sentencia se puede utilizar para hacer que la ejecución regrese de nuevo al llamador de la función.

REGLA

La función **devolver** termina inmediatamente la función en la cual se ejecute.

Por ejemplo, la función:

$$f(x) = \frac{x}{1+x^2}$$

se definirá como:

```
real función F(E real:x)
inicio
  devolver (x/(1+x*x))
fin_función
```

15V

Otro ejemplo puede ser la definición de la función trigonométrica, cuyo valor es

$$\tan(x) = \frac{\sin(x)}{\cos(x)}$$

→ V
abab

donde $\sin(x)$ y $\cos(x)$ son las funciones seno y coseno —normalmente funciones internas—. La declaración de la función es

```
real función tan (E real:x)
//funcion tan igual a sen (x)/cos(x), angulo x en radianes
inicio
  devolver (sen(x)/cos(x))
fin_función
```

UNIBO

Observe que se incluye un comentario para describir la función. Es buena práctica incluir documentación que describa brevemente lo que hace la función, lo que representan sus parámetros o cualquier otra información que explique la definición de la función. En aquellos lenguajes de programación —como Pascal— que exigen sección de declaraciones, éstas se situarán al principio del programa.

Para que las acciones descritas en un subprograma función sean ejecutadas, se necesita que éste sea invocado desde un programa principal o desde otros subprogramas a fin de proporcionarle los argumentos de entrada necesarios para realizar esas acciones.

Los argumentos de la declaración de la función se denominan *parámetros formales*, *ficticios* o *mu-dos* («dummy»); son nombres de variables, de otras funciones o procedimientos y que sólo se utilizan dentro del cuerpo de la función. Los argumentos utilizados en llamada a la función se denominan *pa-rámetros actuales*, que a su vez pueden ser constantes, variables, expresiones, valores de funciones o nombres de funciones o procedimientos.

6.2.2. Invocación a las funciones

Una función puede ser llamada de la forma siguiente:

nombre_función (lista de parametros actuales)

nombre_función

función que llama

lista de parametros actuales

constantes, variables, expresiones, valores de funciones, nom-bres de funciones o procedimientos

Cada vez que se llama a una función desde el algoritmo principal se establece automáticamente una correspondencia entre los parámetros formales y los parámetros actuales. Debe haber exactamente el mis-mo número de parámetros actuales que de parámetros formales en la declaración de la función y se pre-supone una correspondencia uno a uno de izquierda a derecha entre los parámetros formales y los actuales.

Una llamada a la función implica los siguientes pasos:

1. A cada parámetro formal se le asigna el valor real de su correspondiente parámetro actual.
2. Se ejecuta el cuerpo de acciones de la función.
3. Se devuelve el valor de la función al nombre de la función y se retorna al punto de llamada.

Ejemplo 6.1

Definición de la función: $y = x^n$ (potencia n de x)

```

real : función potencia(E real:x;E entero:n)
var
    entero: i, y
inicio
    y ← 1
    desde i ← 1 hasta abs(n) hacer
        y ← y*x
    fin_desde
    si n < 0 entonces
        y ← 1/y
    fin_si
    devolver (y)
fin_función
  
```

abs(n) es la función valor absoluto de n a fin de considerar exponentes positivos o negativos.

Invocación de la función

z ← potencia (2.5, -3)

parámetros actuales

Transferencia de información

```

x = 2.5      n = -3
z = 0.064

```

```

proceso1 → y
escribir(x)
fin

```

Ejemplo 6.2

Función potencia para el cálculo de N elevado a A. El número N deberá ser positivo, aunque podrá tener parte fraccionaria, A es un real.

```

algoritmo Elevar_a_potencia
var
  real : a,n

inicio
  escribir('Deme numero positivo ')
  leer(n)
  escribir('Deme exponente ')
  leer(a)
  escribir('N elevado a =', potencia(n, a))
fin

real función potencia (E real: n, a)
inicio
  devolver(EXP(a * LN(n)))
fin_función

```

```

1 → 1
...
fin

```

En este caso

var

Ejemplo 6.3

Diseñar un algoritmo que contenga un subprograma de cálculo del factorial de un número y una llamada al mismo.

Como ya es conocido por el lector el algoritmo factorial, lo indicaremos expresamente.

```

entero función FACTORIAL(E entero:n)
var
  entero: i,f
  //advertencia, segun el resultado, f puede ser real
inicio
  f ← 1
  desde i ← 1 hasta n hacer
    f ← f*i
  fin_desde
  devolver (f)
fin_función

```

.obs:

y el algoritmo que contiene un subprograma de cálculo del factorial de un número y una llamada al mismo

```

algoritmo función_factorial
var entero: x, y, numero

inicio
  escribir ('Deme un numero entero y positivo')
  leer(numero)
  x ← factorial(numero)

```

2.8.0

... 2.8.0

ATE

```

    y ← factorial(5)
    escribir(x, y)
fin

entero función factorial (E entero: n)
var entero: i, f
inicio
    f ← 1
    desde i ← 1 hasta n hacer
        f ← f*i
    fin_desde
devolver(f)
fin_función

```

En este caso los parámetros actuales son: una variable (*número*) y una constante (5).

Ejemplo 6.4

Realizar el diseño de la función $y = x^3$ (cálculo del cubo de un número).

```

algoritmo prueba
var
    entero: N
inicio //Programa principal
N ← cubo(2)
escribir ('2 al cubo es', N)
escribir ('3 al cubo es', cubo(3))
fin

entero función cubo(E entero: x)
inicio
    devolver(x*x*x)
fin_función

```

La salida del algoritmo sería:

```

2 al cubo es 8
3 al cubo es 27

```

Las funciones pueden tener muchos argumentos, pero solamente un resultado: *el valor de la función*. Esto limita su uso, aunque se encuentran con frecuencia en cálculos científicos. Un concepto más potente es el proporcionado por el subprograma procedimiento que se examina en el siguiente apartado.

Ejemplo 6.5

Algoritmo que contiene y utiliza unas funciones (seno y coseno) a las que les podemos pasar el ángulo en grados.

```

algoritmo Sen_cos_en_grados
var real : g

```

```

inicio
  escribir('Deme ángulo en grados')
  leer(g)
  escribir(seno(g))
  escribir(coseno(g))
fin

```

```

real función coseno (E real : g)
inicio
  devolver(COS(g*2*3.141592/360))
fin_función

```

```

real: función seno (E real g)
inicio
  devolver( SEN(g*2*3.141592/360))
fin_función

```

Ejemplo 6.6

Algoritmo que simplifique un quebrado, dividiendo numerador y denominador por su máximo común divisor.

```

algoritmo Simplificar_quebrado
var
  entero : n, d

inicio
  escribir('Deme numerador')
  leer(n)
  escribir('Deme denominador')
  leer(d)
  escribir(n, '/', d, '=', n div mcd(n, d), '/', d div mcd(n, d))
fin

  entero función mcd (E entero: n, d)
  var
    entero : r
  inicio
    r ← n MOD d
    mientras r <> 0 hacer
      n ← d
      d ← r
      r ← n MOD d
    fin_mientras
    devolver(d)
  fin_función

```

Ejemplo 6.7

Supuesto que nuestro compilador no tiene la función seno. Podríamos calcular el seno de x mediante la siguiente serie:

$$\text{sen}(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \text{ (hasta 17 términos)}$$

x (ángulo en radianes).

El programa nos tiene que permitir el cálculo del seno de ángulos en grados mediante el diseño de una función seno(x), que utilizará, a su vez, las funciones potencia(x,n) y factorial(n), que también deberán ser implementadas en el algoritmo.

Se terminará cuando respondamos N (no) a la petición de otro ángulo.

algoritmo Calcular_seno

var real : gr

carácter : resp

inicio

repetir

escribir('Deme ángulo en grados')

leer(gr)

escribir('Seno(', gr, ') = ', seno(gr))

escribir('¿Otro ángulo?')

leer(resp)

hasta_que resp = 'N'

fin

real función factorial (**E** entero:n)

var

real : f

entero : i

inicio

$f \leftarrow 1$

desde $i \leftarrow 1$ **hasta** n **hacer**

$f \leftarrow f * i$

fin_desde

devolver(f)

fin_función

real función potencia (**E** real:x; **E** entero:n)

var real : pot

entero : i

inicio

$pot \leftarrow 1$

desde $i \leftarrow 1$ **hasta** n **hacer**

$pot \leftarrow pot * x$

fin_desde

devolver(pot)

fin_función

real función seno (**E** real:gr)

var real : x, s

entero : i, n

inicio

$x \leftarrow gr * 3.141592 / 180$

$s \leftarrow x$

```

desde i ← 2 hasta 17 hacer
    n ← 2 * i - 1
    si i MOD 2 <> 0 entonces
        s ← s - potencia(x, n) / factorial(n)
    si_no
        s ← s + potencia(x, n) / factorial(n)
    fin_si
fin_desde
devolver(s)
fin_función

```

6.3. PROCEDIMIENTOS (SUBROUTINAS)

Aunque las funciones son herramientas de programación muy útiles para la resolución de problemas, su alcance está muy limitado. Con frecuencia se requieren subprogramas que calculen varios resultados en vez de uno solo, o que realicen la ordenación de una serie de números, etc. En estas situaciones la *función* no es apropiada y se necesita disponer del otro tipo de subprograma: el *procedimiento* o *subrutina*.

Un *procedimiento* o *subrutina*¹ es un subprograma que ejecuta un proceso específico. Ningún valor está asociado con el nombre del procedimiento; por consiguiente, no puede ocurrir en una expresión. Un procedimiento se llama escribiendo su nombre, por ejemplo, SORT, para indicar que un procedimiento denominado SORT se va a usar. Cuando se invoca el procedimiento, los pasos que lo definen se ejecutan y a continuación se devuelve el control al programa que le llamó.

Procedimiento *versus* función

Los procedimientos y funciones son subprogramas cuyo diseño y misión son similares; sin embargo, existen unas diferencias esenciales entre ellos.

1. Un procedimiento es llamado desde el algoritmo o programa principal mediante su nombre y una lista de parámetros actuales, o bien con la instrucción **llamar_a** (**call**). Al llamar al procedimiento se detiene momentáneamente el programa que se estuviera realizando y el control pasa al procedimiento llamado. Después que las acciones del procedimiento se ejecutan, se regresa a la acción inmediatamente siguiente a la que se llamó.
2. Las funciones devuelven un valor, los procedimientos pueden devolver 0, 1 o *n* valores y en forma de lista de parámetros.
3. El procedimiento se declara igual que la función, pero su nombre no está asociado a ninguno de los resultados que obtiene.

La declaración de un procedimiento es similar a la de funciones.

```

procedimiento nombre [(lista de parámetros formales)]
    <acciones>
fin_procedimiento

```

¹ En FORTRAN, la subrutina representa el mismo concepto que procedimiento. No obstante, en la mayor parte de los lenguajes el término general para definir un subprograma es procedimiento o simplemente subprograma.

Los parámetros formales tienen el mismo significado que en las funciones; los parámetros variables —en aquellos lenguajes que los soportan, por ejemplo, Pascal— están precedidos cada uno de ellos por la palabra **var** para designar que ellos obtendrán resultados del procedimiento en lugar de los valores actuales asociados a ellos.

El procedimiento se llama mediante la instrucción

```
[llamar_a] nombre [(lista de parámetros actuales)]
```

La palabra **llamar_a** (**call**) es opcional y su existencia depende del lenguaje de programación.

El ejemplo siguiente ilustra la definición y uso de un procedimiento para realizar la división de dos números y obtener el cociente y el resto.

Variables enteras:

```
DIVIDENDO
DIVISOR
COCIENTE
```

Procedimiento

```
procedimiento division (E entero:Dividendo,Divisor; S entero: Cociente, Resto)
```

```
  inicio
    Cociente ← Dividendo DIV Divisor
    Resto ← Dividendo-Cociente*Divisor
  fin_procedimiento
```

Algoritmo principal

```
algoritmo aritmética
var
  entero: M, N, P, Q, S, T
inicio
  leer(M, N)
  llamar_a division (m, N, P, Q)
  escribir(P, Q)
  llamar_a division (M*N-4, N+1,S, T)
  escribir(S, T)
fin
```

6.3.1. Sustitución de argumentos/parámetros

La lista de parámetros, bien *formales* en el procedimiento o *actuales* (reales) en la llamada se conocen como *lista de parámetros*.

```
procedimiento demo
.
.
.
fin_procedimiento
```

o bien

```
procedimiento demo (lista de parametros formales)
```

is

y la instrucción llamadora

```
llamar_a demo (lista de parametros actuales)
```

s

Cuando se llama al procedimiento, cada parámetro formal toma como valor inicial el valor del correspondiente parámetro actual. En el ejemplo siguiente se indican la sustitución de parámetros y el orden correcto.

```
algoritmo demo
```

```
//definición del procedimiento
```

```
entero: años
```

```
real: numeros, tasa
```

```
inicio
```

```
...
```

```
llamar_a calculo(numero, años, tasa)
```

```
...
```

```
fin
```

```
procedimiento calculo($ real: p1; E entero: p2; E real: p3)
```

```
inicio
```

```
p3 ... p1 ... p2 ... p2
```

```
fin_procedimiento
```

Las acciones sucesivas a realizar son las siguientes:

1. Los parámetros reales sustituyen a los parámetros formales.
2. El cuerpo de la declaración del procedimiento se sustituye por la llamada del procedimiento.
3. Por último, se ejecutan las acciones escritas por el código resultante.

Ejemplo de procedimiento 6.8

Algoritmo que transforma un número introducido por teclado en notación decimal a romana. El número será entero y positivo y no excederá de 3.000.

SIN UTILIZAR PROGRAMACIÓN MODULAR

```
algoritmo romanos
```

```
var entero : n, digito, r, j
```

```
inicio
```

```
repetir
```

```
escribir('Deme número')
```

```
leer(n)
```

```
hasta_que (n >= 0) Y (n <= 3000)
```

```
r ← n
```

```
digito ← r DIV 1000
```

```
r ← r MOD 1000
```

```
desde j ← 1 hasta digito hacer
```

```
escribir('M')
```

is

r → osipib

= osipib is

rdidibae

os is

b is

s

```

fin_desde
digito ← r DIV 100
r ← r MOD 100
si digito = 9 entonces
    escribir('C', 'M')
si_no
    si digito > 4 entonces
        escribir('D')
        desde j ← 1 hasta digito - 5 hacer
            escribir('C')
        fin_desde
    si_no
        si digito = 4 entonces
            escribir('C', 'D')
        si_no
            desde j ← 1 hasta digito hacer
                escribir('C')
            fin_desde
        fin_si
    fin_si
fin_si
digito ← r DIV 10
r ← r MOD 10
si digito = 9 entonces
    escribir('X', 'C')
    si_no
        si digito > 4 entonces
            escribir('L')
            desde j ← 1 hasta digito - 5 hacer
                escribir('X')
            fin_desde
        si_no
            si digito = 4 entonces
                escribir('X', 'L')
            si_no
                desde j ← 1 hasta digito hacer
                    escribir('X')
                fin_desde
            fin_si
        fin_si
    fin_si
digito ← r
si digito = 9 entonces
    escribir('I', 'X')
si_no
    si digito > 4 entonces
        escribir('V')
        desde j ← 1 hasta digito - 5 hacer
            escribir('I')
        fin_desde
    si_no
        si digito = 4 entonces
            escribir('I', 'V')

```

```

    si_no
        desde j ← 1 hasta digito hacer
            escribir('I')
        fin_desde
    fin_si
fin_si
fin_si
fin

```

MEDIANTE PROGRAMACIÓN MODULAR

```

algoritmo Romanos
var entero : n,r,digito

```

```

inicio
    repetir
        escribir('Deme número')
        leer(n)
        hasta_que (n >= 0) Y (n <= 3000)
            r ← n
            digito ← r div 1000
            r ← r MOD 1000
            calccifrarom(digito, 'M', ' ', ' ')
            digito ← r div 100
            r ← r MOD 100
            calccifrarom(digito, 'C', 'D', 'M')
            digito ← r div 10
            r ← r MOD 10
            calccifrarom(digito, 'X', 'L', 'C')
            digito ← r
            calccifrarom(digito, 'I', 'V', 'X')
    fin

```

```

procedimiento calccifrarom(E entero: digito; E caracter: v1,v2,v3)

```

```

var entero: j
inicio
    si digito = 9 entonces
        escribir( v1, v3)
    si_no
        si digito > 4 entonces
            escribir(v2)
            desde j ← 1 hasta digito - 5 hacer
                escribir(v1)
            fin_desde
        si_no
            si digito = 4 entonces
                escribir(v1, v2)
            si_no
                desde j ← 1 hasta digito hacer
                    escribir(v1)
                fin_desde
        fin_si
    fin_si
fin_procedimiento

```

6.4. ÁMBITO: VARIABLES LOCALES Y GLOBALES

Las variables utilizadas en los programas principales y subprogramas se clasifican en dos tipos:

- *variables locales*
- *variables globales.*

Una *variable local* es aquella que está declarada y definida dentro de un subprograma, en el sentido de que está dentro de ese subprograma y es distinta de las variables con el mismo nombre declaradas en cualquier parte del programa principal. *El significado de una variable se confina al procedimiento en el que está declarada.* Cuando otro subprograma utiliza el mismo nombre se refiere a una posición diferente en memoria. Se dice que tales variables son *locales* al subprograma en el que están declaradas.

Una *variable global* es aquella que está declarada para el programa o algoritmo principal, del que dependen todos los subprogramas.

La parte del programa/algoritmo en que una variable se define se conoce como *ámbito* (*scope*, en inglés).

El uso de variables locales tiene muchas ventajas. En particular, hace a los subprogramas independientes, con la comunicación entre el programa principal y los subprogramas manipulados estructuralmente a través de la lista de parámetros. Para utilizar un procedimiento sólo necesitamos conocer lo que hace y no tenemos que estar preocupados por su diseño, es decir, cómo están programados.

Esta característica hace posible dividir grandes proyectos en piezas más pequeñas independientes. Cuando diferentes programadores están implicados, ellos pueden trabajar independientemente.

A pesar del hecho importante de los subprogramas independientes y las variables locales, la mayoría de los lenguajes proporcionan algún método para tratar ambos tipos de variables.

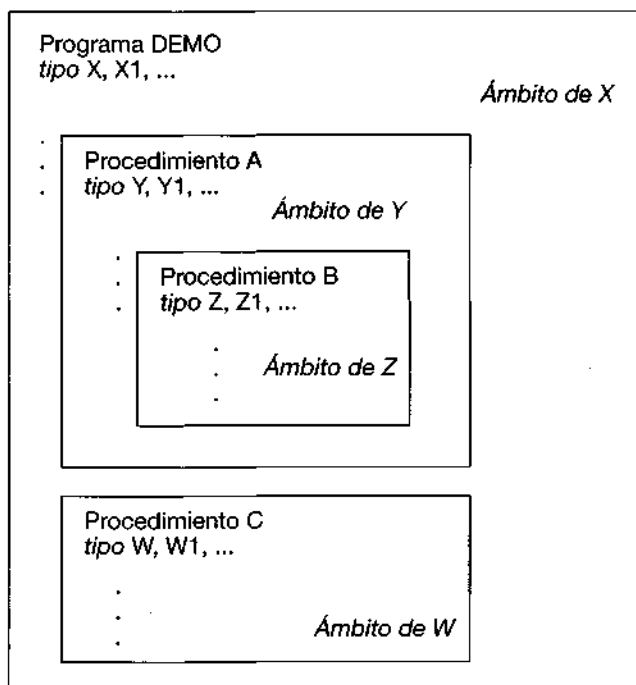


Figura 6.4. Ámbito de identificadores.

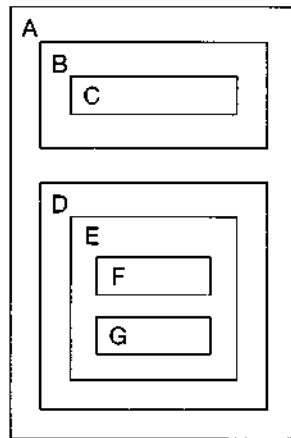
Una variable local a un subprograma no tiene ningún significado en otros subprogramas. Si un subprograma asigna un valor a una de sus variables locales, este valor no es accesible a otros programas, es decir, no pueden utilizar este valor. A veces, también es necesario que una variable tenga el mismo nombre en diferentes subprogramas.

Por el contrario, las variables globales tienen la ventaja de compartir información de diferentes subprogramas sin una correspondiente entrada en la lista de parámetros.

En un programa sencillo con un subprograma, cada variable u otro identificador es o bien local al procedimiento o global al programa completo. Sin embargo, si el programa incluye procedimientos que engloban a otros procedimientos —*procedimientos anidados*—, entonces la noción de global/local es algo más complicado de entender.

El *ámbito* de un identificador (variables, constantes, procedimientos) es la parte del programa donde se conoce el identificador. Si un procedimiento está definido localmente a otro procedimiento, tendrá significado sólo dentro del ámbito de ese procedimiento. A las variables les sucede lo mismo; si están definidas localmente dentro de un procedimiento, su significado o uso se confina a cualquier función o procedimiento que pertenezca a esa definición.

La Figura 6.5 muestra un esquema de un programa con diferentes procedimientos, algunas variables son locales y otras globales. En la citada figura se muestra el ámbito de cada definición.



Variables
definidas en

Accesibles desde

A	A, B, C, D, E, F, G
B	B, C
C	C
D	D, E, F, G
E	E, F, G
F	F
G	G

que se

A.2 = x

de pa-

405

Figura 6.5. Ámbito de definición de variables.

Los lenguajes que admiten variables locales y globales suelen tener la posibilidad explícita de definir dichas variables como tales en el cuerpo del programa, o, lo que es lo mismo, definir su ámbito de actuación, para ello se utilizan las cabeceras de programas y subprogramas, con lo que se definen los ámbitos.

Las variables definidas en un ámbito son accesibles en el mismo, es decir, en todos los procedimientos interiores.

Ejemplo 6.9

La función (signo) realiza la siguiente tarea: dado un número real x , si x es 0, entonces se devuelve un 0; si x es positivo, se devuelve 1, y si x es negativo, se devuelve un valor -1.

La declaración de la función es

```
entero función signo(E real: x)
var entero: s
```

```

inicio
  //valores de signo: +1,0,-1
  si x = 0 entonces s ← 0
  si x > 0 entonces s ← 1
  si x < 0 entonces s ← -1
  devolver (s)
fin_función

```

Antes de llamar a la función, la variable (S), como se declara dentro del subprograma, es local al subprograma y sólo se conoce dentro del mismo. Veamos ahora un pequeño algoritmo donde se invoque la función.

```

algoritmo SIGNOS
var
  entero: a, b, c
  real: x, y, z
inicio
  x ← 5.4
  a ← signo(x)
  y ← 0
  b ← signo(y)
  z ← 7.8975
  c ← signo(z-9)
  escribir('Las respuestas son', a, ' ', b, ' ', c)
fin

```

Si se ejecuta este algoritmo, se obtienen los siguientes valores:

x = 5.4	x es el parámetro actual de la primera llamada a <i>signo(x)</i>
a = signo(5.4)	a toma el valor 1
y = 0	
b = signo(0)	b toma el valor de 0
z = 7.8975	
c = signo(7.8975-9)	c toma el valor -1

La línea escrita al final será:

Las respuestas son 1 0 -1

Ejemplo 6.10

```

algoritmo DEMOX
var entero: A, X, Y
inicio
  x ← 5
  A ← 10
  y ← F(x)
  escribir (x, A, y)
fin

```

```

entero función F(E entero: N)
var
  entero: X
inicio
  A ← 5
  X ← 12
  devolver (N+A)
fin_función

```

A la variable global A se puede acceder desde el algoritmo y desde la función. Sin embargo, X identifica a dos variables distintas: una local al algoritmo y sólo se puede acceder desde él y otra local a la función.

Al ejecutar el algoritmo se obtendrían los siguientes resultados:

```

X = 5
A = 10
Y = F(5)
A = 5
X = 12
F = 5+5 = 10
Y = 10

```

invocación a la función F(N) se realiza un paso del parámetro actual X al parámetro formal N.

se modifica el valor de A en el algoritmo principal por ser A global.

no se modifica el valor de X en el algoritmo principal porque X es local.

se pasa el valor del argumento X(5) a través del parámetro N.

se escribirá la línea

```
5 5 10
```

ya que X es el valor de la variable local X en el algoritmo; A, el valor de A en la función, ya que se pasa este valor al algoritmo; Y es el valor de la función F(X).

6.5. COMUNICACIÓN CON SUBPROGRAMAS: PASO DE PARÁMETROS

Cuando un programa llama a un subprograma, la información se comunica a través de la lista de parámetros y se establece una correspondencia automática entre los parámetros formales y actuales. *Los parámetros actuales son «sustituídos» o «utilizados» en lugar de los parámetros formales.*

La declaración del subprograma se hace con

```

procedimiento nombre (clase tipo_de_dato: F1;
                      clase tipo_de_dato: F2;
                      .....
                      clase tipo_de_dato :Fn)
.
.
.
fin_procedimiento

```

y la llamada al subprograma con

```
llamar_a nombre (A1, A2, ..., An)
```

donde $F_1, F_2, \dots, F_n$ son los parámetros formales y $A_1, A_2, \dots, A_n$ los parámetros actuales o reales.

Las clases de parámetros podrían ser:

- (E) Entrada
- (S) Salida
- (E/S) Entrada/Salida

Existen dos métodos para establecer la correspondencia de parámetros:

1. *Correspondencia posicional.* La correspondencia se establece apareando los parámetros reales y formales según su posición en las listas: así, F_i se corresponde con A_i , donde $i = 1, 2, \dots, n$. Este método tiene algunas desventajas de legibilidad cuando el número de parámetros es grande.
2. *Correspondencia por el nombre explícito,* también llamado *método de paso de parámetros por nombre.* En este método, en las llamadas se indica explícitamente la correspondencia entre los parámetros reales y formales. Este método se utiliza en Ada. Un ejemplo sería:

```
SUB(Y => B, X => 30);
```

que hace corresponder el parámetro actual B con el formal Y, y el parámetro actual 30 con el formal X durante la llamada de SUB.

Por lo general, la mayoría de los lenguajes usan exclusivamente la correspondencia posicional y ese será el método empleado en este libro.

Las cantidades de información que pueden pasarse como parámetros son *datos de tipos simples, estructurados* —en los lenguajes que admiten su declaración— y *subprogramas*.

6.5.1. Paso de parámetros

Existen diferentes métodos para la *transmisión o el paso de parámetros* a subprogramas. Es preciso conocer el método adoptado por cada lenguaje, ya que la elección puede afectar a la semántica del lenguaje. Dicho de otro modo, un mismo programa puede producir diferentes resultados bajo diferentes sistemas de paso de parámetros.

Los parámetros pueden ser clasificados como:

- entradas:* las entradas proporcionan valores desde el programa que llama y que se utilizan dentro de un procedimiento. En los subprogramas función, las entradas son los argumentos en el sentido tradicional;
- salidas:* las salidas producen los resultados del subprograma; de nuevo si se utiliza el caso una función, éste devuelve un valor calculado por dicha función, mientras que con procedimientos pueden calcularse cero, una o varias salidas;
- entradas/salidas:* un solo parámetro se utiliza para mandar argumentos a un programa y para devolver resultados.

Desgraciadamente, el conocimiento del tipo de parámetros no es suficiente para caracterizar su funcionamiento; por ello, examinaremos los diferentes métodos que se utilizan para pasar o transmitir parámetros.

Los métodos más empleados para realizar el paso de parámetros son:

- *paso por valor* (también conocido por *parámetro valor*),
- *paso por referencia o dirección* (también conocido por *parámetro variable*),

- *paso por nombre*,
- *paso por resultado*.

6.5.2. Paso por valor

El paso por valor se utiliza en muchos lenguajes de programación; por ejemplo, C, Modula-2, Pascal, Algol y Snobol. La razón de su popularidad es la analogía con los argumentos de una función, donde los valores se proporcionan en el orden de cálculo de resultados. Los parámetros se tratan como variables locales y los valores iniciales se proporcionan copiando los valores de los correspondientes argumentos.

Los parámetros formales —locales a la función— reciben como valores iniciales los valores de los parámetros actuales y con ello se ejecutan las acciones descritas en el subprograma.

No se hace diferencia entre un argumento que es variable, constante o expresión, ya que sólo importa el valor del argumento. La Figura 6.6 muestra el mecanismo de paso por valor de un procedimiento con tres parámetros.

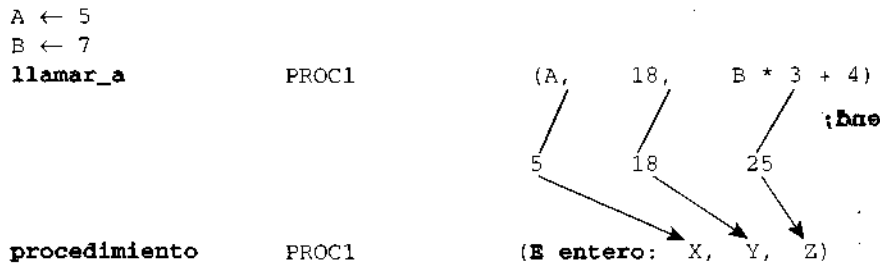


Figura 6.6. Paso por valor.

El mecanismo de paso se resume así:

Valor primer parámetro: $A = 5$.

Valor segundo parámetro: $\text{constante} = 18$.

Valor tercer parámetro: expresión $B * 3 + 4 = 25$.

Los valores 5, 18 y 25 se transforman en los parámetros X, Y, Z respectivamente cuando se ejecuta el procedimiento.

Aunque el *paso por valor* es sencillo, tiene una limitación acusada: *no existe ninguna otra conexión con los parámetros actuales*, y entonces los cambios que se produzcan por efecto del subprograma no producen cambios en los argumentos originales y, por consiguiente, no se pueden pasar valores de retorno al punto de llamada: es decir, todos los parámetros son sólo de entrada. El parámetro actual no puede modificarse por el subprograma. Cualquier cambio realizado en los valores de los parámetros formales durante la ejecución del subprograma se destruye cuando se termina el subprograma.

La llamada por valor no devuelve información al programa que llama.

Existe una variante de la llamada por valor y es la llamada por *valor resultado*. Las variables indicadas por los parámetros formales se inicializan en la llamada al subprograma por valor tras la ejecución del subprograma; los resultados (valores de los parámetros formales) se transfieren a los actuales. Este método se utiliza en algunas versiones de FORTRAN.

6.5.3. Paso por referencia

En numerosas ocasiones se requiere que ciertos parámetros sirvan como parámetros de salida, es decir, se devuelvan los resultados a la unidad o programas que llama. Este método se denomina *paso por*

referencia o también de *llamada por dirección o variable*. La unidad que llama pasa a la unidad llamada la dirección del parámetro actual (que está en el ámbito de la unidad llamante). Una referencia al correspondiente parámetro formal se trata como una referencia a la posición de memoria, cuya dirección se ha pasado. Entonces una variable pasada como parámetro real es compartida, es decir, se puede modificar directamente por el subprograma.

Este método existe en FORTRAN, COBOL, Modula-2, Pascal, PL/1 y Algol 68. La característica de este método se debe a su simplicidad y su analogía directa con la idea de que las variables tienen una posición de memoria asignada desde la cual se pueden obtener o actualizar sus valores.

El área de almacenamiento (direcciones de memoria) se utiliza para pasar información de entrada y/o salida; en ambas direcciones.

En este método los parámetros son de entrada/salida y los parámetros se denominan *parámetros variables*.

Los parámetros valor y parámetros variable se suelen definir en la cabecera del subprograma. En el caso de lenguajes como Pascal, los parámetros variables deben ir precedidos por la palabra clave **var**;

```

program muestra;
//parametros actuales a y b, c y d paso por referencia
  procedure prueba(var x,y:integer);
    begin //procedimiento
//proceso de los valores de x e y
    end;

begin
.
.
.
1. prueba(a, c);
.
.
.
2. prueba(b, d);
.
.
.
end.

```

La primera llamada en (1) produce que los parámetros *a* y *c* sean sustituidos por *x* e *y* si los valores de *x* e *y* se modifican dentro de *a* o *c* en el algoritmo principal. De igual modo, *b* y *d* son sustituidos por *x* e *y*, y cualquier modificación de *x* o *y* en el procedimiento afectará también al programa principal.

La llamada por referencia es muy útil para programas donde se necesita la comunicación del valor en ambas direcciones.

NOTAS

Ambos métodos de paso de parámetros se aplican tanto a la llamada de funciones como a las de procedimientos:

- Una función tiene la posibilidad de devolver los valores al programa principal de dos formas: a) como valor de la función, b) por medio de argumentos gobernados por la llamada de referencia en la correspondencia parámetro actual-parámetro formal.
- Un procedimiento sólo puede devolver valores por el método de devolución de resultados.

El lenguaje Pascal permite que el programador especifique el tipo de paso de parámetros y, en un mismo subprograma, unos parámetros se pueden especificar por valor y otros por referencia.

```

procedure Q(i:integer; var j:integer);
begin
  i := i+10;
  j := j+10;
  write(i, j)
end;

```

Los parámetros formales son *i, j*, donde *i* se pasa por valor y *j* por referencia.

6.5.4. Comparaciones de los métodos de paso de parámetros

Para examinar de modo práctico los diferentes métodos, consideremos un ejemplo único y veamos los diferentes valores que toman los parámetros. El algoritmo correspondiente con un procedimiento SUBR:

```

algoritmo DEMO
var
  entero: A,B,C
inicio //DEMO
  A ← 3
  B ← 5
  C ← 17
  llamar_a SUBR(A, A, A+B, C)
  escribir(C)
fin //DEMO

procedimiento SUBR (<Modo> entero: x, y;
  E entero:z; <Modo> entero: v)
inicio
  x ← x+1
  v ← y+z
fin_procedimiento

```

Modo por valor

a) sólo por valor

no se transmite ningún resultado, por consiguiente
C no varía C = 17

b) valor_resultado

A = 3
B = 5
C = 17

pasa al procedimiento

x = A = 3
y = A = 3
z = A + B = 8
v = C = 17

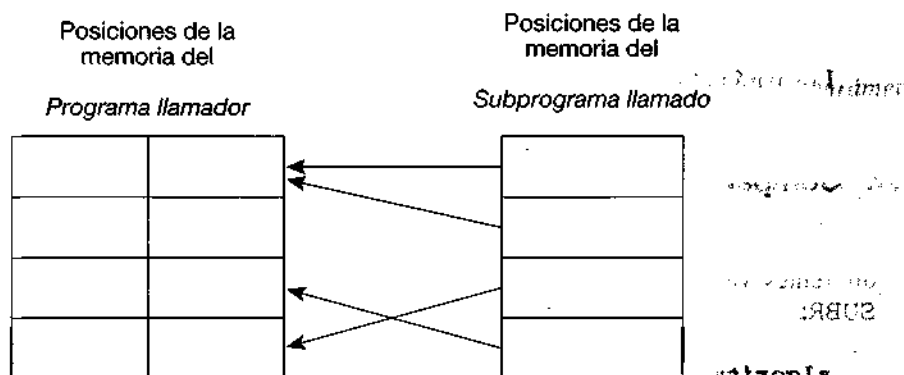
al ejecutar el procedimiento quedará

$$x = x + 1 = 3 + 1 = 4$$

$$v = y + z = 3 + 8 = 11$$

el parámetro llamado *v* pasa el valor del resultado *v* a su parámetro actual correspondiente, *C*.
Por tanto, *C* = 11.

Modo por referencia



C recibirá el valor 12.

Utilizando variables globales

```

algoritmo DEMO
var entero: A, B, C
inicio
  A ← 3
  B ← 5
  C ← 17
  llamar_a SUBR
  escribir (C)
fin

```

```

procedimiento SUBR
inicio
  a ← a+1
  c ← a+a+b
fin_procedimiento

```

Es decir, el valor de *C* será 13.

La llamada por referencia es el sistema estándar utilizado por FORTRAN para pasar parámetros. La llamada por nombre es estándar en Algol 60. Simula 67 proporciona llamadas por valor, referencia y nombre.

Pascal permite pasar bien por valor bien por referencia

```

procedure demo(y:integer; var z:real);

```

especifica que *y* se pasa por valor mientras que *z* se pasa por referencia —indicado por la palabra reservada **var**—. La elección entre un sistema u otro puede venir determinado por diversas consideraciones, como evitar efectos laterales no deseados provocados por modificaciones inadvertidas de parámetros formales (véase Apartado 6.7).

6.5.5. Síntesis de la transmisión de parámetros

Los métodos de transmisión de parámetros más utilizados son *por valor* y *por referencia*.

El paso de un parámetro por valor significa que el valor del argumento —*parámetro actual* o *real*— se asigna al parámetro formal. En otras palabras, antes de que el subprograma comience a ejecutarse, el argumento se evalúa a un valor específico (por ejemplo, 8 o 12). Este valor se copia entonces en el correspondiente parámetro formal dentro del subprograma.

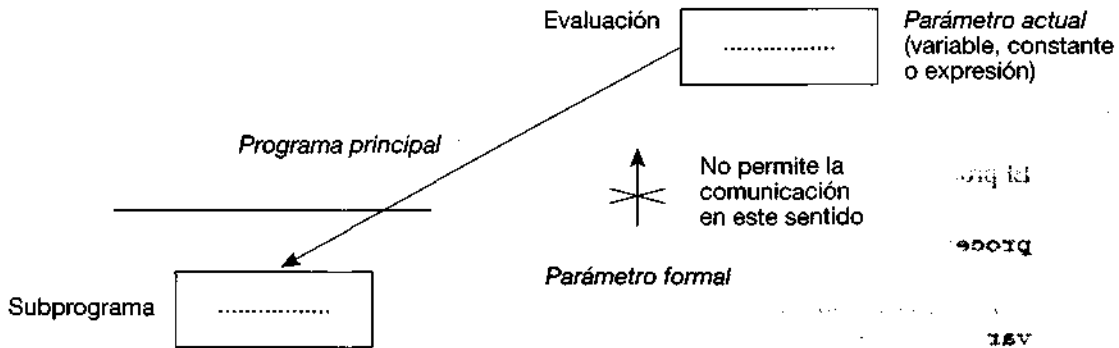


Figura 6.7. Paso de un parámetro por valor.

Una vez que el procedimiento arranca, cualquier cambio del valor de tal parámetro formal no se refleja en un cambio en el correspondiente argumento. Esto es, cuando el subprograma se termine, el argumento actual tendrá exactamente el mismo valor que cuando el subprograma comenzó, con independencia de lo que haya sucedido al parámetro formal. Este método es el método por defecto en Pascal si no se indica explícitamente otro. Estos parámetros de entrada se denominan *parámetros valor*. En los algoritmos indicaremos como $\langle \text{modo} \rangle E$ (entrada). El paso de un *parámetro por referencia* o *dirección* se llama *parámetro variable*, en oposición al parámetro por valor. En este caso, la posición o dirección (no el valor) del argumento o parámetro actual se envía al subprograma. Si a un parámetro formal se le da el atributo de parámetro variable —en Pascal con la palabra reservada **var**— y si el parámetro actual es una variable, entonces un cambio en el parámetro formal se refleja en un cambio en el correspondiente parámetro actual, ya que ambos tienen la misma posición de memoria.

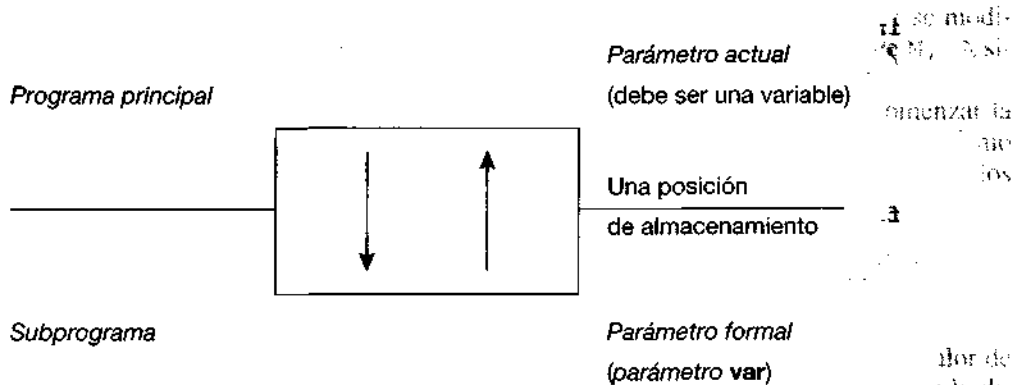


Figura 6.8. Paso de un parámetro por referencia.

Para indicar que deseamos transmitir un parámetro por dirección, lo indicaremos con la palabra *parámetro variable* —en Pascal se indica con la palabra reservada **var**— y especificaremos como $\langle \text{modo} \rangle E/S$ (entrada/salida) o S (salida).

Ejemplo 6.11

Se trata de realizar el cálculo del área de un círculo y la longitud de la circunferencia en función del valor del radio leído desde el teclado.

Recordemos las fórmulas del área del círculo y de la longitud de la circunferencia:

$$A = \pi \cdot r^2 = \pi \cdot r \cdot r$$

$$C = 2\pi \cdot r = 2 \cdot \pi \cdot r \quad \text{donde } \pi = 3.141592$$

Los parámetros de entrada: radio

Los parámetros de salida: área, longitud

El procedimiento *círculo* calcula los valores pedidos.

```

procedimiento circulo(E real: radio; S real: area, longitud)
  //parametros valor: radio
  //parametros variable: area, longitud
var
  real: pi
inicio
  pi ← 3.141592
  area ← pi*radio*radio
  longitud ← 2*pi*radio
fin_procedimiento

```

Los parámetros formales son: radio, área, longitud, de los cuales son valor (radio) y variable (área, longitud).

Invocemos el procedimiento *círculo* utilizando la instrucción

```

llamar_a circulo(6, A, C)

```

```

  //(programa principal
inicio
  //llamada al procedimiento
  llamar_a circulo(6, A, C)
  .
  .
fin

procedimiento circulo(E real: radio; S real: area, longitud)
  //parametros valor:radio
  //parametros variable: area, longitud

inicio
  pi ← 3.141592
  area ← pi*radio*radio
  longitud ← 2*pi*radio
fin_procedimiento

```

Ejemplo 6.12

Consideremos un subprograma N con dos parámetros formales: *i*, transmitido por valor, y *j*, por variable.

```

algoritmo M
//variables A, B enteras
var
    entero: A, B

inicio
    A ← 2
    B ← 3
    llamar_a N(A,B)
    escribir(A, B)
fin //algoritmo M

procedimiento N(E entero: I; E/S entero: j)
//parametros valor i
//parametros variable j
inicio
    i ← i+10
    j ← j+10
    escribir(i, j)
fin_procedimiento

```

Si se ejecuta el procedimiento N, veamos qué resultados se escribirán:

A y B son parámetros actuales
i y j son parámetros formales.

Como *i* es por valor, se transmite el valor de A a *i*, es decir, $i = A = 2$. Cuando *i* se modifica por defecto de $i \leftarrow i+10$ a 12, A no cambia y, por consiguiente, a la terminación de N, A sigue valiendo 2.

El parámetro B se transmite por referencia, es decir, *j* es un parámetro variable. Al comenzar la ejecución de N, B se almacena como el valor *j* y cuando se suma 10 al valor de *j*, *i* en sí mismo no cambia. El valor del parámetro B se cambia a 13. Cuando los valores *i*, *j* se escriben en N, los resultados son:

12 y 13

pero cuando retornan a M y al imprimir los valores de A y B, sólo ha cambiado el valor B. El valor de $i = 12$ se pierde en N cuando éste ya termina. El valor de *j* también se pierde, pero éste es la dirección, no el valor 13.

Se escribirá como resultado final de la instrucción **escribir**(A, B):

6.6. FUNCIONES Y PROCEDIMIENTOS COMO PARÁMETROS

Hasta ahora los subprogramas que hemos considerado implicaban dos tipos de parámetros formales: *parámetros valor* y *parámetros variable*. Sin embargo, en ocasiones se requiere que un procedimiento o función dado invoque a otro procedimiento o función que ha sido definido fuera del ámbito de ese procedimiento o función. Por ejemplo, se puede necesitar que un procedimiento P invoque la función F que puede estar o no definida en el procedimiento P; esto puede conseguirse transfiriendo como parámetro el procedimiento o función externa (F) o procedimiento o función dado (por ejemplo, el P). En resumen, algunos lenguajes de programación —entre ellos Pascal— admiten *parámetros procedimiento* y *parámetros función*.

Ejemplos

```

procedimiento P(E func: F1; E real: x,y)
real función F(E func: F1,F2; E entero:x,y)

```

Los parámetros formales del procedimiento P son la función F1 y las variables X e Y, y los parámetros formales de la función F son las funciones F1 y F2, y las variables x e y.

Procedimientos función

Para ilustrar el uso de los parámetros función, consideremos la función *integral* para calcular el área bajo una curva $f(x)$ para un intervalo $a \leq x \leq b$.

La técnica conocida para el cálculo del área es subdividir la región en rectángulos, como se muestra en la Figura 6.9, y sumar las áreas de los rectángulos. Estos rectángulos se construyen subdividiendo el intervalo $[a, b]$ en m subintervalos iguales y formando rectángulos con estos subintervalos como bases y alturas dadas por los valores de f en los puntos medios de los subintervalos.

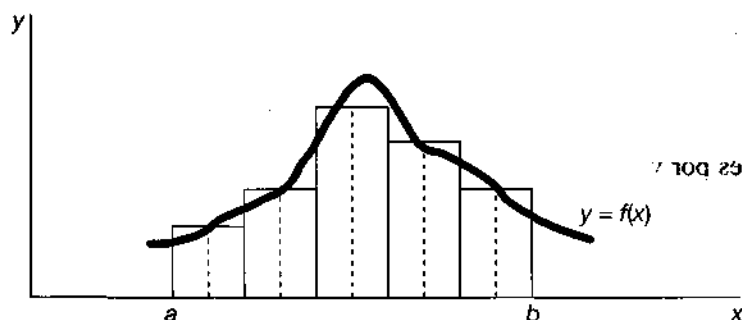


Figura 6.9. Cálculo del área bajo la curva $f(x)$.

La función *integral* debe tener los parámetros formales a , b y n , que son parámetros valor ordinarios actuales de tipo real; se asocian con los parámetros formales a y b ; un parámetro actual de tipo entero —las subdivisiones— se asocia con el parámetro formal n y una función actual se asocia con el parámetro formal f .

Los parámetros función se designan como tales con una cabecera de función dentro de la lista de parámetros formales. La función *integral* podrá definirse por

```

real función integral(E func: f; E real: a,b; E entero: n)
el tipo func-tipo real func función (E real: x)

```

aquí la función $f(x)$: real especifica que f es una función parámetro que denota una función cuyo parámetro formal y valor son de tipo real. El correspondiente parámetro función actual debe ser una función que tiene un parámetro formal real y un valor real. Por ejemplo, si integrado es una función de valor real con un parámetro y tipo real función (E real:x):func

Area $\leftarrow$ Integral(Integrado, 0, 1.5, 20)

es una referencia válida a función.

Diseñar un algoritmo que utilice la función Integral para calcular el área bajo el gráfico de las funciones $f_1(x) = x^3 - 6x^2 + 10x$ e. Integrado por $\text{Integrado}(x) = x^2 + 3x + 2$ para $0 \leq x \leq 4$.

algoritmo Area_bajo_curvas

tipo

real función(E real : x) : func

var

real : a,b

entero : n

inicio

escribir('¿Entre qué límites?')

leer(a, b)

escribir('¿Subintervalos?')

leer(n)

escribir(integral(f1, a, b, n))

escribir(integral(integrado, a, b, n))

fin

real FUNCIÓN f1 (E real : x)

inicio

devolver(x * x * x - 6 * x * x + 10 * x)

fin_funcion

real FUNCIÓN integrado (E real : x)

inicio

devolver(x * x + 3 * x + 2)

fin_función

real FUNCIÓN integral (E func : f; E real : a, b; E entero : n)

var

real : baserectangulo, altura, x, s

entero : i

inicio

baserectangulo $\leftarrow$ (b - a) / n

x $\leftarrow$ a + baserectangulo/2

s $\leftarrow$ 0

desde i $\leftarrow$ 1 **hasta** n **hacer**

altura $\leftarrow$ f(x)

s $\leftarrow$ s + baserectangulo * altura

x $\leftarrow$ x + baserectangulo

fin_desde

devolver(s)

fin_función

6.7. LOS EFECTOS LATERALES

Las modificaciones que se produzcan mediante una función o procedimiento en los elementos situados fuera del subprograma (función o procedimiento) se denominan *efectos laterales*. Aunque en algunos casos los efectos laterales pueden ser beneficiosos en la programación, es conveniente no recurrir a ellos de modo general. Consideramos a continuación los efectos laterales en funciones y en procedimientos.

6.7.1. En procedimientos

La comunicación del procedimiento con el resto del programa se debe realizar normalmente a través de parámetros. Cualquier otra comunicación entre el procedimiento y el resto del programa se conoce como *efectos laterales*. Como ya se ha comentado, los efectos laterales son perjudiciales en la mayoría de los casos, como se indica en la Figura 6.10.

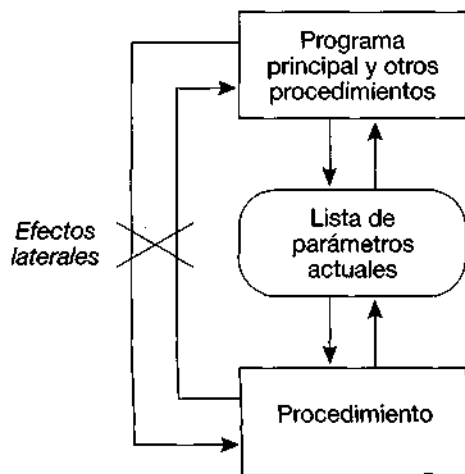


Figura 6.10. Efectos laterales en procedimientos.

Si un procedimiento modifica una variable global (distinta de un parámetro actual), éste es un *efecto lateral*. Por ello, excepto en contadas ocasiones, no debe aparecer en la declaración del procedimiento. Si se necesita una variable temporal en un procedimiento, utilice una variable local, no una variable global. Si se desea que el programa modifique el valor de una variable global, utilice un parámetro formal variable en la declaración del procedimiento y a continuación utilice la variable global como el parámetro actual en una llamada al procedimiento.

En general, se debe seguir la regla de «ninguna variable global en procedimientos», aunque esta prohibición no significa que los procedimientos no puedan manipular variables globales. De hecho, el cambio de variables globales se deben pasar al procedimiento como parámetros actuales. Las variables globales no se deben utilizar directamente en las instrucciones en el cuerpo de un procedimiento; en su lugar, utilice un parámetro formal o variable local.

En aquellos lenguajes en que es posible declarar constantes —como Pascal— se pueden utilizar constantes globales en una declaración de procedimiento; la razón reside en el hecho de que las constantes no pueden ser modificadas por el procedimiento y, por consiguiente, no existe peligro de que se puedan modificar inadvertidamente.

6.7.2. En funciones

Una función toma los valores de los argumentos y devuelve *un único valor*. Sin embargo, al igual que los procedimientos, una función —en algunos lenguajes de programación— puede hacer cosas similares a un procedimiento o subrutina. Una función puede tener parámetros variables además de parámetros valor en la lista de parámetros formales. Una función puede cambiar el contenido de una variable global y ejecutar instrucciones de entrada/salida (escribir un mensaje en la pantalla, leer un valor del teclado, etc.). Estas operaciones se conocen como *parámetros laterales* y se deben evitar.

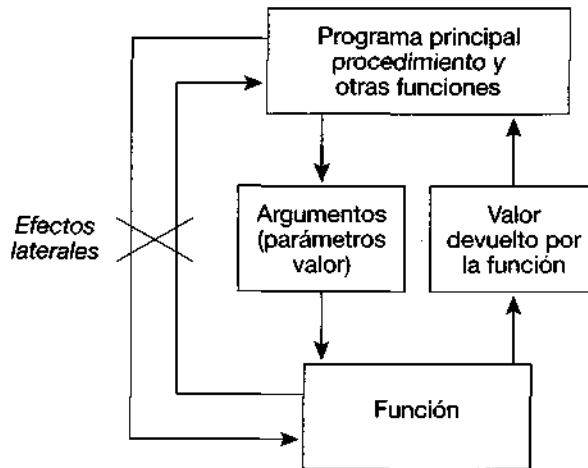


Figura 6.11. Efectos laterales en una función.

Los efectos laterales están considerados —normalmente— como una mala técnica de programación, pues hacen más difícil de entender los programas.

Toda la información que se transfiere entre procedimientos y funciones debe realizarse a través de la lista de parámetros y no a través de variables globales. Esto convertirá al procedimiento o función en módulos independientes que pueden ser comprobados y depurados por sí solos, lo que evitará preocuparnos por el resto de las partes del programa.

6.8. RECURSIÓN (RECURSIVIDAD)

Como ya se conoce, un subprograma puede llamar a cualquier otro subprograma y éste a otro, y así sucesivamente; dicho de otro modo, los subprogramas se pueden anidar. Se puede tener

```
A llamar_a B, B llamar_a C, C llamar_a D
```

Cuando se produce el retorno de los subprogramas a la terminación de cada uno de ellos el proceso resultante será

```
D retornar_a C, C retornar_a B, B retornar_a A
```

¿Qué sucedería si dos subprogramas de una secuencia son los mismos?

```
A llamar_a A
```